# Crime Management System

**Submitted by**

*Sumanyu Seth – 25MCI10040*

*Lakshay Aggarwal - 25MCI10047*

*Ajay Rakwal - 25MCI10329*

*Nikhil Kumar - 25MCI10036*

*Aryan - 25MCI10082*

**in partial fulfillment for the award of the degree of**

**Master of Computer Applications**

**IN**

Artificial Intelligence & Machine Learning


**Chandigarh University**

**Submitted to**

**Mr. Shalabh Bhatiya(E18471)**

# ACKNOWLEDGEMENT

I would like to express my deep sense of gratitude to my respected faculty members and the Department of Computer Applications for their guidance, encouragement, and continuous support throughout the completion of this project titled “Crime Management System.”

I deem it a pleasure to acknowledge my sincere gratitude to my project guide, Mr. Shalabh Bhatiya (Assistant Professor, UIC), under whose supervision this project was carried out. His insightful guidance, constructive feedback, and timely suggestions provided continuous motivation and played a crucial role in the successful completion of this work.

I would also like to extend my sincere thanks to the Head of the Department and all the faculty members for creating a supportive academic environment and providing valuable resources that contributed to the successful execution of this project.

I am grateful to my classmates and friends for their constant support, helpful suggestions, and encouragement during the development and testing phases of the project. Their feedback helped in improving the overall functionality and usability of the system. Finally, I would like to acknowledge the open-source community and developers for providing the tools and technologies such as Python, Tkinter, and MySQL, which made the implementation of this project possible. This project has significantly enhanced my understanding of database management, application development, and real-world problem-solving.

# 1. INTRODUCTION

The Crime Management System is a database-driven desktop application developed to assist law enforcement agencies in efficiently managing and organizing crime-related information. In traditional systems, maintaining records of crimes, suspects, witnesses, and evidence is often done manually or through fragmented digital tools, which can lead to inefficiencies, data redundancy, and lack of proper coordination. This project aims to overcome these limitations by providing a centralized and structured system for handling crime data.

The application is developed using Python with Tkinter for the graphical user interface and MySQL as the backend database. It follows a modular design where the user interface interacts with the database through a well-defined logic layer. This ensures smooth data flow, easy maintenance, and scalability of the system.

The system allows authorized officers to log in securely and access a dashboard where they can perform various operations such as adding new crime records, updating existing data, and managing related entities like suspects, witnesses, and evidence. Each of these components is interconnected through relational database design, ensuring consistency and integrity of data.

A key aspect of the system is the implementation of CRUD (Create, Read, Update, Delete) operations along with relational constraints such as foreign keys and cascading deletes. This ensures that all related data remains synchronized; for example, deleting a crime record automatically removes its associated evidence and witness details.

Overall, the Crime Management System demonstrates the practical application of database management concepts in solving real-world problems. It provides a structured, secure, and efficient way to manage crime-related information while improving data accessibility and reliability.

# 2. LITERATURE REVIEW

Database Management Systems (DBMS) play a crucial role in modern information systems, especially in domains that require structured data storage, retrieval, and integrity. In the field of law enforcement, efficient data management is essential for tracking criminal activities, maintaining records, and ensuring quick access to critical information. Traditional methods of

record-keeping, often manual or semi-digital, are prone to errors, redundancy, and lack of coordination, which highlights the need for automated database-driven solutions.

The concept of centralized crime management systems has been widely studied, where relational databases are used to organize data into structured tables such as crimes, suspects, witnesses, and evidence. These systems rely on relational database principles, including normalization, primary keys, and foreign key constraints, to maintain data consistency and eliminate redundancy. The use of such structured approaches ensures that all related information remains interconnected and easily accessible.

Another important aspect in database applications is the implementation of CRUD (Create, Read, Update, Delete) operations, which form the backbone of any data management system. These operations allow users to efficiently manage records while ensuring that updates are reflected across all related entities. In systems dealing with sensitive data, such as crime records, maintaining data integrity and consistency is of utmost importance.

User authentication and access control are also critical components in modern database systems. Secure login mechanisms ensure that only authorized personnel can access or modify the data, thereby protecting sensitive information from unauthorized use. This is particularly important in law enforcement systems where data confidentiality is a priority.

The Crime Management System project builds upon these established concepts by integrating a relational database with a user-friendly interface and modular program structure. It demonstrates how database design principles, secure authentication, and structured data operations can be combined to develop a practical and efficient system for managing crime-related information.

# 3. REQUIREMENTS

## 3.1 Functional Requirements

- **User Authentication:** The system should provide a secure login mechanism for officers using valid credentials, ensuring that only authorized personnel can access the system.

- **Crime Record Management:** The application must allow users to perform CRUD operations on crime records, including adding new cases, viewing existing records, updating details, and deleting entries.
- **Witness Management:** The system should support storing and managing multiple witnesses associated with a particular crime, along with their statements.
- **Suspect Tracking:** The system must maintain detailed records of suspects linked to crimes, including their descriptions and relevant information.
- **Evidence Management:** The application should allow users to record and track physical evidence, including details such as description, location found, and recovery date.
- **Search and Filtering:** Users should be able to search crime records based on attributes such as type or location for quick retrieval of information.

### 3.2 Non-Functional Requirements

- **Performance:** The system should respond quickly to user actions, especially during data retrieval and updates, even with increasing data volume.
- **Data Integrity:** The database must enforce constraints such as primary keys and foreign keys to ensure consistency and eliminate redundancy.
- **Security:** Sensitive data should be protected through secure authentication mechanisms and controlled access.
- **Usability:** The graphical user interface should be simple, intuitive, and easy to navigate for users with minimal technical knowledge.
- **Maintainability:** The system should follow a modular structure, making it easier to update, debug, and extend in the future.
- **Portability:** The application should run smoothly on systems that support Python and MySQL without requiring complex configurations.

## 4. METHODOLOGY / IMPLEMENTATION

The development of the Crime Management System follows a structured and modular approach, focusing on efficient data handling, clear separation of components, and ease of user interaction. The system is designed as a desktop-based application where the graphical user interface, application logic, and database layer work together to manage crime-related information effectively.

The implementation begins with the design of the database schema. A relational database is created using MySQL, where different entities such as crimes, suspects, witnesses, evidence, and users are represented as separate tables. Relationships between these tables are established using foreign keys, ensuring that all related data remains connected and consistent. For example, each crime record can be associated with multiple witnesses and pieces of evidence, forming a structured and normalized database.

Once the database is designed, the application logic is implemented using Python. A centralized module is used to handle database connectivity, ensuring that all components can interact with the database efficiently. The business logic layer manages CRUD operations, executing SQL queries to insert, retrieve, update, and delete records as required. This separation of logic from the interface improves maintainability and readability of the code.

The graphical user interface is developed using Tkinter, which provides an interactive environment for users to interact with the system. The interface includes login screens, dashboards, and data entry forms that allow users to perform operations without needing direct access to the database. Special attention is given to usability, ensuring that navigation between different sections is smooth and intuitive.

The overall implementation can be summarized through the following key steps:

- **Database Design:** Creation of tables with proper relationships and constraints to maintain data integrity.
- **Backend Logic Development:** Implementation of Python modules to handle database operations and business logic.
- **GUI Development:** Design of user-friendly interfaces for data entry, viewing records, and navigation.
- **Integration:** Connecting the GUI with backend logic and database to ensure seamless data flow.
- **Testing:** Verifying system functionality through various test cases, including edge cases such as invalid inputs and deletion of linked records.

This methodology ensures that the system is well-structured, reliable, and capable of handling real-world scenarios efficiently. It also demonstrates the practical application of database management and software development principles in building a functional system.

# 5. SYSTEM ARCHITECTURE

The Crime Management System follows a modular and layered architecture that ensures clear separation between user interaction, application logic, and data management. This design approach improves the system's scalability, maintainability, and overall performance, making it suitable for handling structured crime-related data efficiently.

The system is primarily divided into three interconnected layers:

- **Presentation Layer (GUI):** This layer is responsible for user interaction and is implemented using Tkinter. It provides interfaces such as the login window, dashboard, and data entry forms. Users can perform operations like adding crime records, viewing details, and managing suspects, witnesses, and evidence through this layer. The interface is designed to be intuitive, allowing users to interact with the system without requiring technical knowledge.
- **Application Layer (Logic Layer):** This layer acts as the core processing unit of the system. It handles all business logic and coordinates between the user interface and the database. Modules written in Python manage operations such as authentication, validation of inputs, and execution of CRUD operations. This layer ensures that all user actions are properly processed before interacting with the database.
- **Data Layer (Database):** The data layer is implemented using MySQL and is responsible for storing all information related to users, crimes, suspects, witnesses, and evidence. It uses relational database concepts such as primary keys, foreign keys, and constraints to maintain data integrity and consistency. Cascading operations ensure that related records are automatically updated or deleted when required.

The overall workflow of the system can be understood as follows:

- The user interacts with the GUI to perform an action (e.g., add a crime record)
- The request is passed to the application layer for processing
- The application layer executes the appropriate database query
- The database performs the operation and returns the result
- The result is displayed back to the user through the interface

This layered architecture ensures smooth communication between components while maintaining a clear structure. It also allows future enhancements, such as adding new modules or improving the interface, without affecting the entire system.

# 6. CODE SNIPPET

## 6.1 Code for DB Operations:

### 6.1.1 authenticate_officer:

```
def authenticate_officer(email, password):
    conn = get_db_connection()
    try:
        cur = conn.cursor()
            cur.execute("SELECT officer_id, name FROM users WHERE email=%s AND password=%s", (email, password))
        row = cur.fetchone()
        return row
    finally:
        cur.close()
        conn.close()
```

**# --- Crimes ---**

### 6.1.2 get_all_crimes:

```
def get_all_crimes():
    conn = get_db_connection()
    try:
        cur = conn.cursor(dictionary=True)
```

```
        cur.execute("""
            SELECT c.id, c.type, c.description, c.date_reported, c.location, c.status, c.officer_id,
u.name AS officer_name
            FROM crimes c
            LEFT JOIN users u ON c.officer_id = u.officer_id
            ORDER BY c.date_reported DESC
        """)
        rows = cur.fetchall()
        return rows
    finally:
        cur.close()
        conn.close()
```

**6.1.3 search_crimes:**

```
def search_crimes(keyword):
    kw = f"%{keyword}%"
    conn = get_db_connection()
    try:
        cur = conn.cursor(dictionary=True)
        cur.execute("""
            SELECT c.id, c.type, c.description, c.date_reported, c.location, c.status, c.officer_id,
u.name AS officer_name
            FROM crimes c
            LEFT JOIN users u ON c.officer_id = u.officer_id
            WHERE c.type LIKE %s OR c.location LIKE %s
```

```
            ORDER BY c.date_reported DESC
        """, (kw, kw))
        return cur.fetchall()
    finally:
        cur.close()
        conn.close()
```

**6.1.4 add_crime:**

```
def add_crime(type_, description, date_str, location, officer_id, status='reported'):
    conn = get_db_connection()
    try:
        cur = conn.cursor()
        try:
            date_obj = datetime.datetime.strptime(date_str, "%Y-%m-%d").date()
        except Exception:
            date_obj = None
        cur.execute(
            "INSERT INTO crimes (type, description, date_reported, location, officer_id, status)
VALUES (%s, %s, %s, %s, %s, %s)",
            (type_, description, date_obj, location, officer_id, status)
        )
        conn.commit()
        return cur.lastrowid
    except Exception as e:
        conn.rollback()
```

```
        raise
    finally:
        cur.close()
        conn.close()
```

### 6.1.5 update_crime:

```
def update_crime(crime_id, type_, description, date_str, location, status):
    conn = get_db_connection()
    try:
        cur = conn.cursor()
        try:
            date_obj = datetime.datetime.strptime(date_str, "%Y-%m-%d").date()
        except Exception:
            date_obj = None
        cur.execute(
            "UPDATE crimes SET type=%s, description=%s, date_reported=%s, location=%s, status=%s WHERE id=%s",
            (type_, description, date_obj, location, status, crime_id)
        )
        conn.commit()
        return cur.rowcount > 0
    except Exception:
        conn.rollback()
        raise
    finally:
```

```
        cur.close()
        conn.close()
```

### 6.1.6 delete_crime:

```
def delete_crime(crime_id):
    conn = get_db_connection()
    try:
        cur = conn.cursor()
        cur.execute("DELETE FROM crimes WHERE id=%s", (crime_id,))
        conn.commit()
        return cur.rowcount > 0
    except Exception:
        conn.rollback()
        raise
    finally:
        cur.close()
        conn.close()
```

## 6.2 Code For Dashboard:

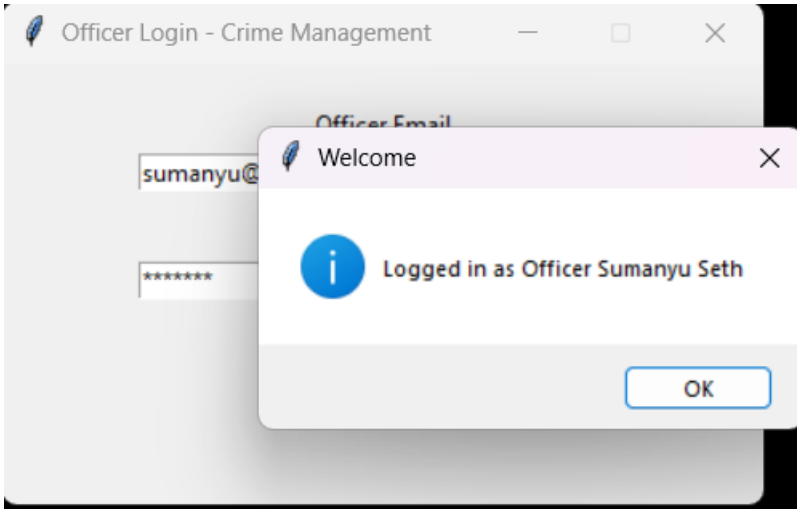
```
def show_dashboard():
    root = tk.Tk()
    root.title("Crime Management Dashboard")
    root.geometry("1000x600")
```

```
    header = tk.Frame(root)
    header.pack(fill=tk.X, padx=10, pady=6)
    tk.Label(header, text=f"Officer: {LOGGED_OFFICER['name']}", font=("Helvetica", 11, "bold")).pack(side=tk.LEFT)
    tk.Button(header, text="Logout", command=lambda: logout(root)).pack(side=tk.RIGHT)
    search_frame = tk.Frame(root)
    search_frame.pack(fill=tk.X, padx=10, pady=6)
    tk.Label(search_frame, text="Search (type or location):").pack(side=tk.LEFT, padx=(0,6))
    search_var = tk.StringVar()
    search_entry = tk.Entry(search_frame, textvariable=search_var, width=40)
    search_entry.pack(side=tk.LEFT)

    def do_search():
        kw = search_var.get().strip()
        for r in crime_tree.get_children():
            crime_tree.delete(r)
        rows = ops.search_crimes(kw) if kw else ops.get_all_crimes()
        for r in rows:
                    crime_tree.insert("", "end", values=(r['id'], r['type'], r['description'][:40], r['date_reported'], r['location'], r.get('status',''), r.get('officer_name','')))

    tk.Button(search_frame, text="Search", command=do_search).pack(side=tk.LEFT, padx=6)
        tk.Button(search_frame, text="Show All", command=lambda: [search_var.set(""), do_search()]).pack(side=tk.LEFT)
    # Treeview for crimes
```

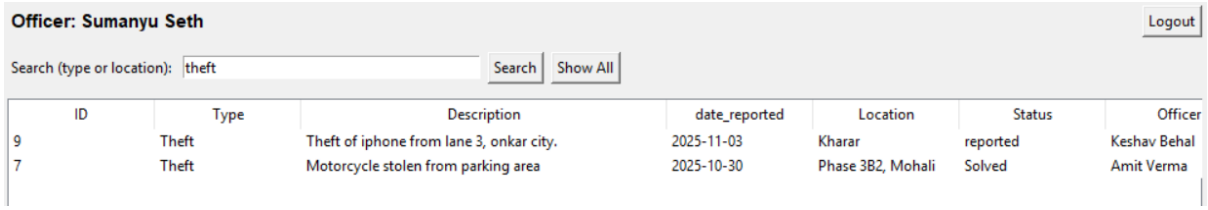
```
cols = ("ID", "Type", "Description", "date_reported", "Location", "Status", "Officer")
crime_tree = ttk.Treeview(root, columns=cols, show="headings", selectmode="browse")
for c in cols:
    crime_tree.heading(c, text=c)
    crime_tree.column(c, anchor="w", width=120)
crime_tree.column("Description", width=300)
crime_tree.pack(fill=tk.BOTH, expand=True, padx=10, pady=6)
```

# 7. LIST OF FIGURES

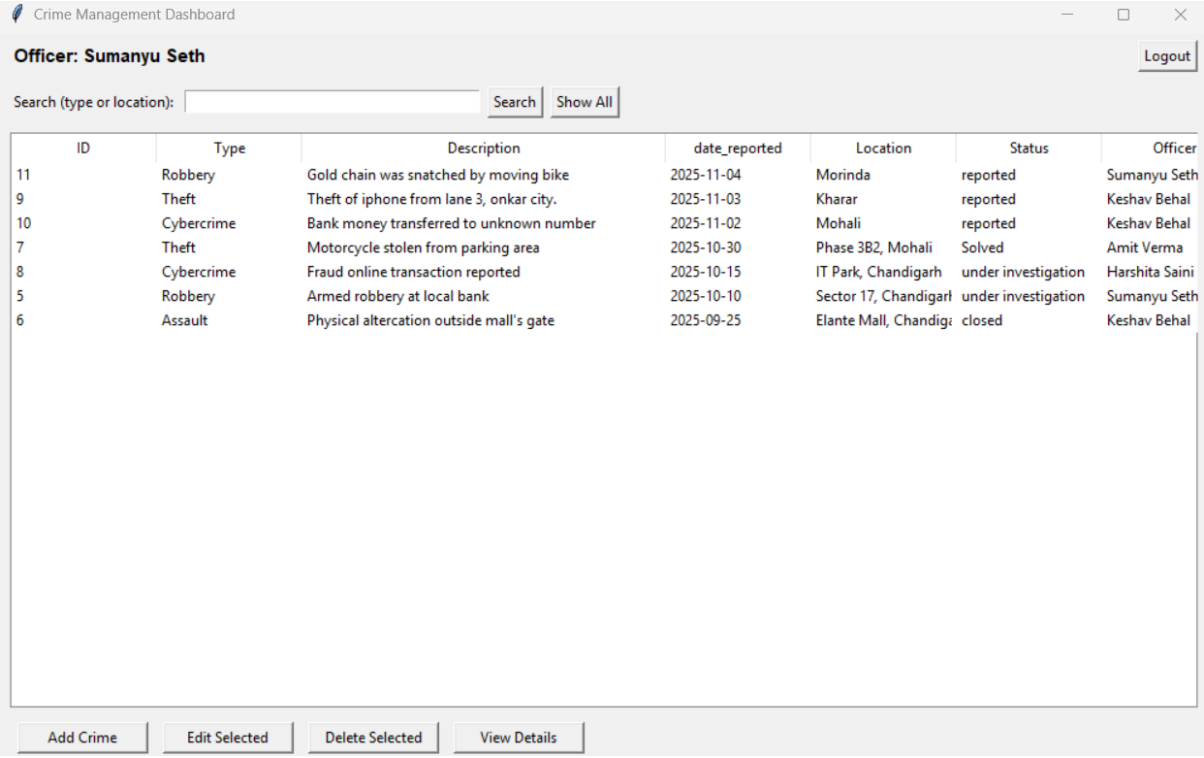
## 7.1 Login page:



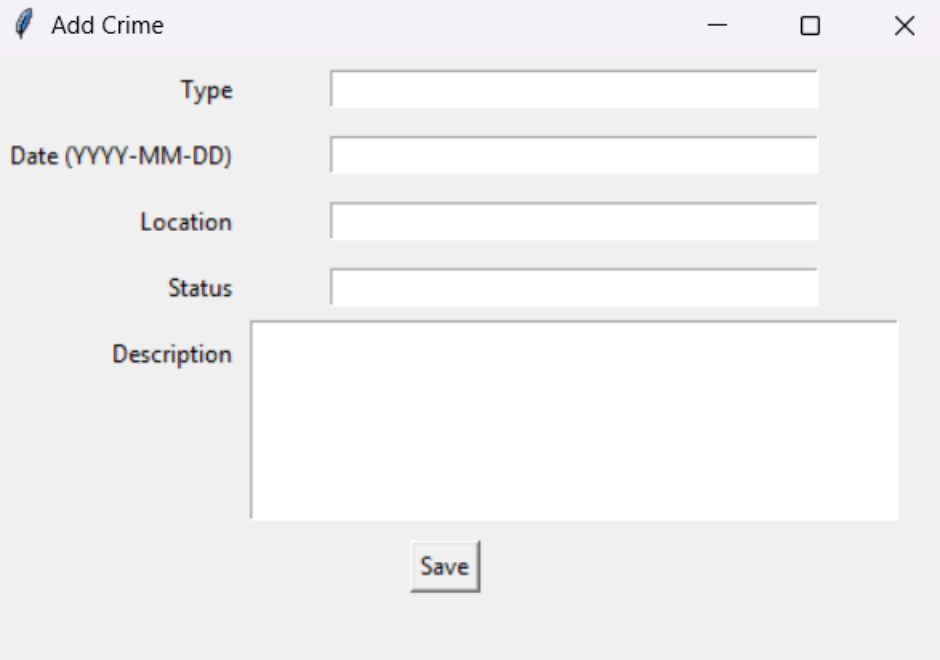

## 7.2 Searching:



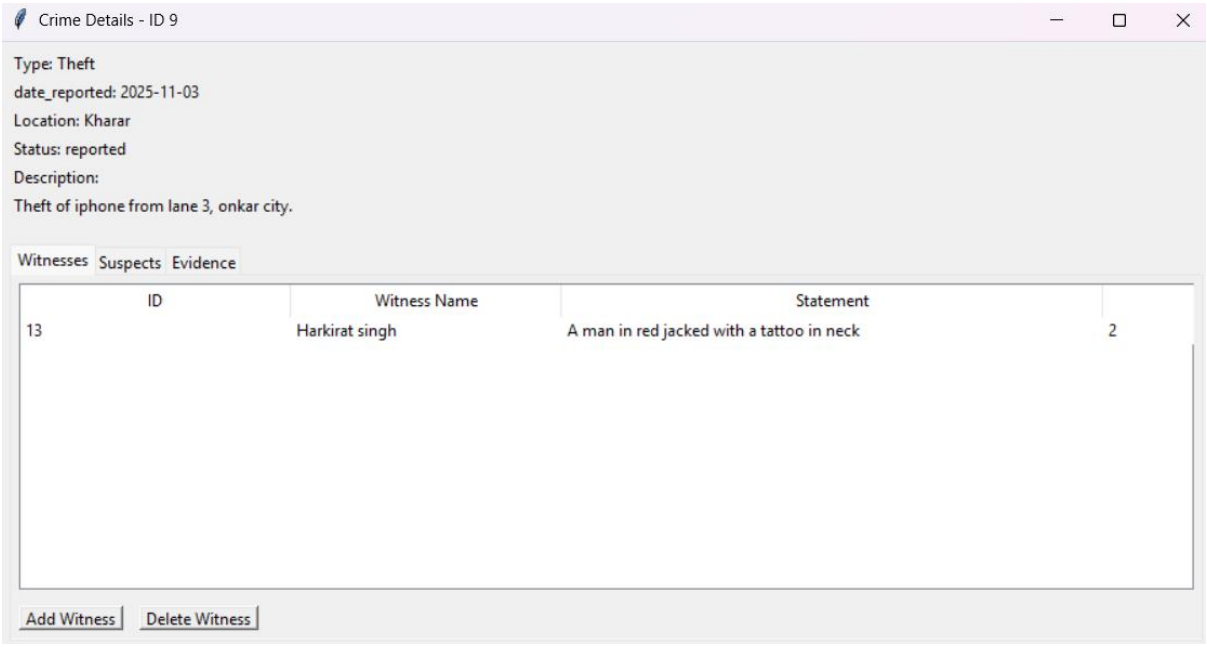
## 7.3 Dashboard:




## 7.4 Add Crime Case Panel:



### 7.5 Crime Details Page:




## 8. CONCLUSION

The Crime Management System successfully demonstrates the practical implementation of database management concepts in a real-world application. The system provides an efficient and structured approach to managing crime-related data, including records of crimes, suspects, witnesses, and evidence, within a centralized platform.

By integrating a graphical user interface with a relational database, the application ensures that users can perform complex data operations in a simple and user-friendly manner. The use of Python and Tkinter enables interactive and responsive interfaces, while MySQL ensures reliable data storage and retrieval. The implementation of CRUD operations allows flexible data management, and the use of relational constraints such as foreign keys helps maintain data integrity and consistency.

One of the key strengths of the system is its modular architecture, which separates the user interface, application logic, and database layers. This design not only improves maintainability but also allows future enhancements without affecting the overall system. Additionally, the inclusion of secure authentication ensures that sensitive data is accessed only by authorized users.

The project highlights important aspects such as data organization, integrity, and efficient retrieval, which are essential in law enforcement systems. It also demonstrates how traditional manual processes can be transformed into automated and reliable digital solutions.

In conclusion, the Crime Management System serves as an effective tool for managing crime data and reflects the importance of database-driven applications in solving real-world problems. It provides a strong foundation for further development and enhancements in the field of information management systems.

## 9. GITHUB LINK

https://github.com/Nikhilkumar0101/25MCI10036_Nikhil_kumar_25MAM1A_Kargil_DBMS/tree/main/DBMS_miniProj

## 10. BIBLIOGRAPHY


- MySQL Documentation https://dev.mysql.com/doc/
- Python Official Documentation https://docs.python.org/3/
- Tkinter Documentation https://docs.python.org/3/library/tkinter.html
- GeeksforGeeks – DBMS Concepts https://www.geeksforgeeks.org/dbms/
- TutorialsPoint – Database Management System https://www.tutorialspoint.com/dbms/index.htm
- W3Schools – SQL Tutorial https://www.w3schools.com/sql/
- Oracle – Introduction to Databases https://www.oracle.com/database/what-is-database/
- IBM – What is DBMS? https://www.ibm.com/topics/dbms